

Transformers and Modern NLP

Technical reference document for Retrieval-Augmented Generation (RAG) practice.

1. Transformer Architecture

Transformers are neural network architectures designed around attention mechanisms. They became foundational for modern natural language processing because they can process relationships between tokens efficiently and model long-range dependencies.

- An encoder transforms an input sequence into contextual representations.
- A decoder generates output tokens autoregressively in decoder-based language models.
- Some architectures use encoders, some use decoders, and some use both.

2. Tokenization

Language models operate on tokens rather than raw strings. A tokenizer converts text into token identifiers and can convert generated token identifiers back into text.

- Tokenization may split words into subword units.
- The vocabulary maps token pieces to integer IDs.
- Special tokens can represent sequence boundaries or control model behavior.

3. Self-Attention

Self-attention allows each token representation to incorporate information from other tokens in the sequence. Queries, keys, and values are transformed from the input representations and combined using attention weights.

- Attention scores determine how strongly one token attends to another.
- Multi-head attention runs several attention projections in parallel.
- Attention masks can restrict which tokens are visible to one another.

4. Positional Information

Attention alone does not inherently encode token order. Transformers therefore add positional information, using learned or fixed positional representations depending on the architecture.

- Position information lets the model distinguish different token orders.
- Modern architectures may use alternative positional encoding methods designed for long contexts.

5. BERT and Encoder Models

BERT is an encoder-based Transformer model designed to produce contextual representations. It became widely used for language understanding tasks such as classification, entity recognition, and extractive question answering.

- Encoder attention can use information from both left and right context.
- BERT-style models are commonly fine-tuned for downstream tasks.

6. GPT and Decoder Models

GPT-style models use decoder Transformer blocks and generate text autoregressively. At each generation step, the model predicts a distribution over the next token given the preceding context.

- Causal masking prevents a token from attending to future tokens during training.
- Generation repeatedly selects or samples the next token until a stopping condition is reached.

7. Hugging Face Transformers

The Hugging Face Transformers ecosystem provides pretrained models, tokenizers, configuration objects, and utilities for inference and fine-tuning.

- A tokenizer prepares text for a model.
- A pretrained model can be loaded for a specific task or adapted to a new task.
- Pipelines provide convenient high-level inference interfaces for common tasks.

8. Fine-Tuning and Inference

Fine-tuning continues training a pretrained model on task-specific data. Inference uses model parameters without updating them.

- Fine-tuning can adapt a general model to a specialized dataset.
- Evaluation data should remain separate from training data.
- Inference performance depends on model size, sequence length, hardware, batching, and generation settings.

9. Transformers and RAG

Transformers can support RAG through embedding models, rerankers, and generation models. A typical RAG system retrieves relevant passages and places them into the context supplied to a Transformer-based language model.

- Embedding models represent documents and queries for semantic search.
- A reranker can reorder retrieved candidates using a stronger relevance model.

- The generation model should distinguish retrieved evidence from the user's question and follow the application's grounding instructions.